

PIA - Proyecto Clasificación de Melanoma

Alberto Piedra Venegas

February 2026

Índice

1. Planteamiento del problema	2
2. Análisis del dataset	2
3. Modelo Fully Connected	3
3.1. Estrategias para combatir el Overfitting y el desbalance de clases	4
3.2. Resultados	4
4. Modelo CNN	5
4.1. Estrategias para combatir el Overfitting y el desbalance de clases	6
4.2. Resultados	6
5. Modelo Transfer Learning	7
5.1. Estrategias para combatir el Overfitting y el desbalance de clases	8
5.2. Resultados	8
6. Comparación entre los modelos	9
6.1. Transfer Learning (TL)	9
6.2. Red Neuronal Convolutacional (CNN)	10
6.3. Fully Connected (FC)	10
7. Conclusiones	10

1. Planteamiento del problema

Uno de los aspectos más críticos para la salud dermatológica es la detección temprana de lesiones cutáneas malignas, por lo que es fundamental contar con sistemas que permitan identificar de manera precisa y rápida posibles casos de melanoma. En este proyecto, nos enfocaremos en el desarrollo de un sistema de clasificación de imágenes dermatoscópicas que permita diferenciar entre lunares benignos y melanomas en etapas iniciales.

Para utilizar estos modelos en la vida real, se podría integrar el software en un dispositivo dermatoscópico portátil o incluso en una aplicación móvil especializada. Esto permitiría que, durante una revisión rutinaria, el personal sanitario o el propio usuario capturen una imagen de la lesión y reciban una evaluación instantánea de riesgo, facilitando una derivación urgente al especialista en caso de detectarse anomalías sospechosas.

2. Análisis del dataset

Para este proyecto utilizaremos un dataset que cuenta con imágenes dermatoscópicas con melanoma y sin melanoma. Este dataset cuenta con +15000 imágenes, separadas entre benignas y malignas.

El propio dataset ya viene con la separación realizada entre entrenamiento y tests.

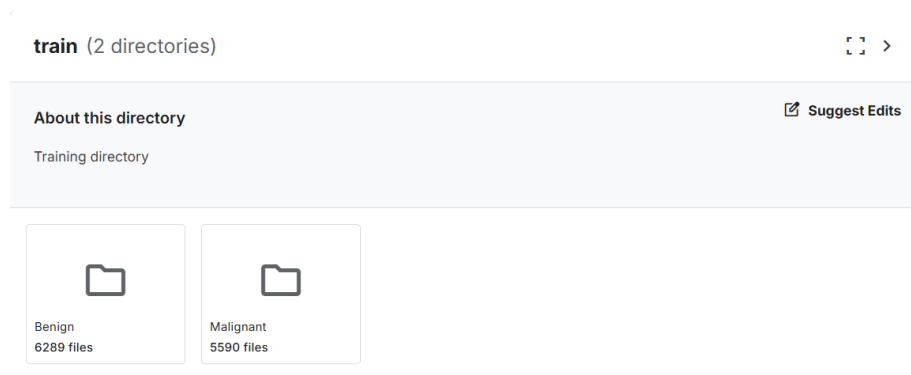


Figura 1: Entrenamiento

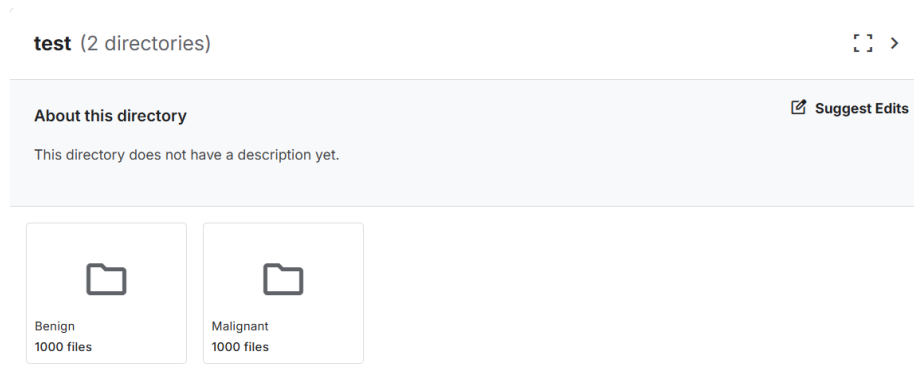


Figura 2: Test

. Este dataset cuenta con un pequeño desbalanceo de clases, contando con 6289 imágenes benignas y 5590 imágenes malignas.

A pesar de que el dataset ya viene separado entre entrenamiento y test, haremos otra separación entre entrenamiento y validación para poder evaluar el rendimiento del modelo.

3. Modelo Fully Connected

El primer modelo que utilizaremos será un modelo denso o también llamado Fully Connected. Este modelo es el más sencillo de los tres, que se utilizarán para este proyecto de clasificación.

Para este modelo primero escalaremos las imágenes a un tamaño de 224x224 píxeles, luego dividiremos el valor de los píxeles entre 255 para que se encuentren en un rango de 0 a 1. A continuación, aplanaremos las imágenes para que puedan ser procesadas por el modelo denso. Este modelo constará de 3 capas densas con una función de activación ReLU y una capa de salida con una función de activación sigmoide que es la que mejor clasifica para 2 categorías.

```
1  modeloFullyConnected = tf.keras.Sequential()
2
3  modeloFullyConnected.add(tf.keras.layers.Rescaling(1./255,
4  input_shape=(224, 224, 3)))
5  modeloFullyConnected.add(tf.keras.layers.Flatten(input_shape
6  =(224, 224,3)))
7  modeloFullyConnected.add(tf.keras.layers.Dense(128, activation=
8  'relu'))
9  modeloFullyConnected.add(tf.keras.layers.Dropout(0.3))
10 modeloFullyConnected.add(tf.keras.layers.Dense(64, activation='
11 relu'))
12 modeloFullyConnected.add(tf.keras.layers.Dense(1, activation='
13 sigmoid'))
14
15 modeloFullyConnected.compile(optimizer='adam', loss='
16 binary_crossentropy', metrics=['recall'])
```

```

12 historyFullyConnected = modeloFullyConnected.fit(X_train,
13 Y_train, epochs=20, validation_data=(X_val, Y_val),
14 class_weight=dict_weights, callbacks = [callback, reduce_lr])

modeloFullyConnected.evaluate(X_test, Y_test)

```

Entrenaremos nuestro modelo con los datos de entrenamiento y validación, para después evaluar su rendimiento con los datos de prueba.

3.1. Estrategias para combatir el Overfitting y el desbalance de clases

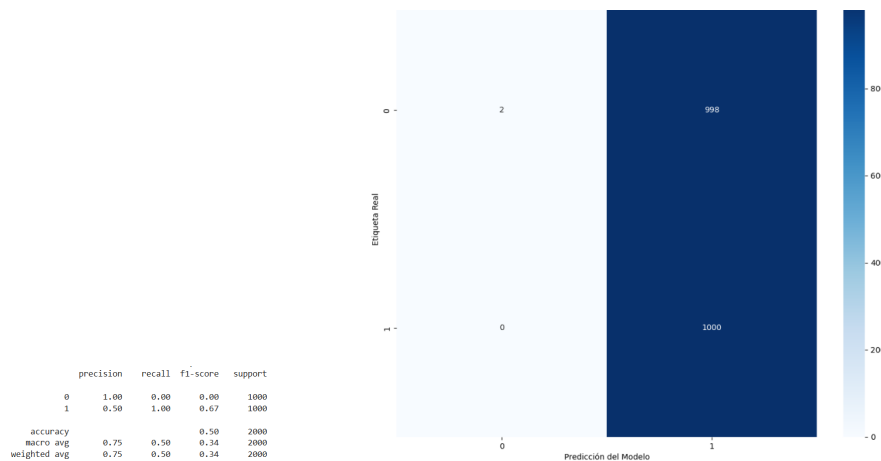
Para combatir el Overfitting utilizaremos la tecnica del Dropout que consiste en desactivar aleatoriamente el 30% de las neuronas de una capa durante el entrenamiento.

Además utilizaremos una función de pesos para combatir el desbalance de clases. Esta función se encarga de penalizar más a las clases con menos imágenes.

Y por último añadiremos un EarlyStopping y un callback para reducir el learning rate cuando el modelo deje de mejorar y para detener el entrenamiento si el modelo no mejora durante unas epocas determinadas..

3.2. Resultados

Tras entrenar el modelo utilizando las estrategias mencionadas anteriormente, obtuvimos los siguientes resultados utilizando los datos de prueba:



Como se puede ver en los resultados, el modelo no da muy buenos resultados y solo predice una misma categoria.

Esto es debido a la perdida de capacidad de discriminación ya que tienen una gran cantidad de parametros ($224 \times 224 \times 3 = 150528$) y a la función de flatten, el modelo prefiere memorizar la clase que mas haya y predecir todas las imagenes de este tipo sin aprender ninguna caracteristica ya que asi sería

la que "mejor resultado" le daría. Sin embargo esto hace que cuando se utilicen imágenes nuevas para probar el modelo, este falle mucho.

4. Modelo CNN

El modelo CNN es un modelo más avanzado que el modelo Fully Connected, ya que utiliza capas convolucionales para extraer características de las imágenes.

Para nuestro modelo, contaremos con 10 capas distintas:

- La primera de ellas servirá como capa de entrada y se encargará también de escalar las imágenes para que los píxeles pasen de un rango de 0 a 255 a un rango de 0 a 1.
- Luego contaremos con 3 bloques de capas convolucionales con una función de activación ReLU, esta capa se encargará de utilizar filtros para detectar distintos patrones y bordes.
- También contaremos con capas de agrupamiento Max Pooling que se encargarán de reducir el tamaño de las imágenes para reducir el cómputo de la imagen.
- Tendremos una capa con el método `flatten()` que se encargará de aplanar las imágenes para que puedan ser procesadas por el modelo denso.
- Tendremos dos capas densas con una función de activación ReLU y una capa de salida con una función de activación Sigmoide, ya que solo tenemos 2 categorías distintas.

```
1  modeloCNN = tf.keras.Sequential()
2
3
4  modeloCNN.add(tf.keras.layers.Rescaling(1./255, input_shape
5  =(224, 224, 3)))
6  modeloCNN.add(Conv2D(filters=32, kernel_size=(3, 3), activation
7  ='relu', input_shape=(224, 224, 3)))
8  modeloCNN.add(MaxPool2D(pool_size=(2, 2)))
9  modeloCNN.add(tf.keras.layers.Dropout(0.5))
10 modeloCNN.add(Conv2D(filters=64, kernel_size=(3, 3), activation
11 ='relu'))
12 modeloCNN.add(MaxPool2D(pool_size=(2, 2)))
13 modeloCNN.add(Conv2D(filters=128, kernel_size=(3, 3),
14 activation='relu'))
15 modeloCNN.add(MaxPool2D(pool_size=(2, 2)))
16 modeloCNN.add(Flatten())
17 modeloCNN.add(Dense(128, activation='relu'))
18 modeloCNN.add(tf.keras.layers.Dropout(0.5))
19 modeloCNN.add(Dense(1, activation='sigmoid'))
20
21 modeloCNN.compile(optimizer='adam', loss='binary_crossentropy',
22 metrics=['recall'])
```

```

19 historyCNN = modeloCNN.fit(X_train, Y_train, epochs=20,
validation_data=(X_val, Y_val), class_weight=dict_weights,
callbacks = [callback,reduce_lr])
20
21 modeloCNN.evaluate(X_test, Y_test)

```

Entrenaremos nuestro modelo con los datos de entrenamiento y validación, para después evaluar su rendimiento con los datos de prueba.

4.1. Estrategias para combatir el Overfitting y el desbalance de clases

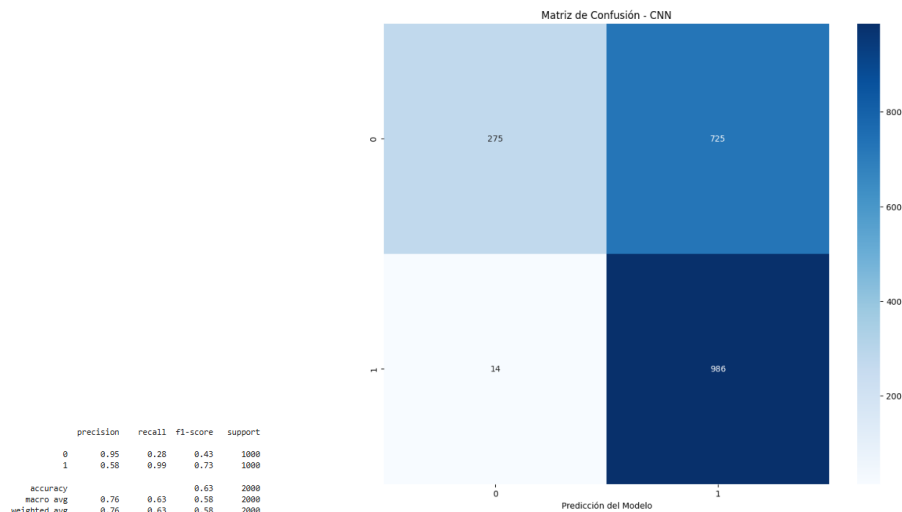
Para combatir el Overfitting utilizaremos la tecnica del Dropout en 2 capas distintas, con una tasa de 0.5 ambas, estas capas se encargarán de desactivar aleatoriamente el 50 % de las neuronas de la capa durante el entrenamiento.

Además utilizaremos una función de pesos para combatir el desbalance de clases. Esta función se encarga de penalizar más a las clases con menos imágenes.

Y por último añadiremos un EarlyStopping y un callback para reducir el learning rate cuando el modelo deje de mejorar y para detener el entrenamiento si el modelo no mejora durante unas epocas determinadas..

4.2. Resultados

Tras entrenar el modelo utilizando las estrategias mencionadas anteriormente, obtuvimos los siguientes resultados utilizando los datos de prueba:



Como se puede ver en los resultados el modelo de CNN da mejores resultados que el modelo Fully Connected, ya que tiene una mayor capacidad de discriminación y puede aprender características más complejas de las imágenes.

Aun así no da los mejores resultados ya que aunque lo hemos entrenado para aumentar la sensibilidad y ha funcionado correctamente para detectar si es maligno con un 0.99 de sensibilidad, para detectar benignos es muy malo con solo un 0.28 de recall.

Es decir el modelo está sobreajustado a la importancia de la clase 1 (Maligno), este modelo puede ser contraproducente ya que el elevado número de falsos positivos comprometería la viabilidad clínica del modelo CNN, a pesar de su alta sensibilidad en la detección de melanomas.

5. Modelo Transfer Learning

El modelo Transfer Learning consiste en utilizar un modelo pre-entrenado en un conjunto de datos grande y utilizarlo como base para nuestro modelo.

Para este proyecto utilizaremos el modelo EfficientNetB1 pre-entrenado en el conjunto de datos ImageNet. Este modelo es una red neuronal convolucional que ha sido entrenada en más de 14 millones de imágenes y ha obtenido una precisión del 84% en el conjunto de datos ImageNet.

Primero cargaremos el modelo pre-entrenado y le añadiremos el parámetro `include_top=False` para que no cargue la capa de salida.

Luego tendremos dos etapas distintas en nuestro modelo:

1. Fase de Calentamiento: En esta fase congelamos el conocimiento de EfficientNetB1 y entrenaremos solo las capas que añadamos nosotros.
2. Fase de Fine-Tuning: En esta fase descongelaremos solo las últimas 40 capas para que el modelo se adapte a nuestro dataset. Aquí también añadiremos capas de GlobalAveragePooling2D para reducir la complejidad de los datos y 2 capas densas, una para razonamiento intermedio y otra con el clasificador final del modelo.

```
1  base_model = EfficientNetB1(weights='imagenet', include_top=
2  False, input_shape=(224, 224, 3))
3  base_model.trainable = False
4
5  inputs = tf.keras.Input(shape=(224, 224, 3))
6  x = base_model(inputs, training=False)
7  x = GlobalAveragePooling2D()(x)
8  x = Dense(256, activation='relu')(x)
9  x = Dropout(0.4)(x)
10
11 predictions = Dense(1, activation='sigmoid')(x)
12 modeloB1 = Model(inputs, predictions)
13
14 modeloB1.compile(optimizer=Adam(learning_rate=1e-3),
15                  loss='binary_crossentropy',
16                  metrics=['recall'])
17
18 history_warmup = modeloB1.fit(
19     X_train, Y_train,
20     epochs=10,
```

```

20     validation_data=(X_val, Y_val),
21     class_weight=dict_weights,
22     batch_size=32,
23     callbacks=[callback, reduce_lr]
24 )
25
26 base_model.trainable = True
27 for layer in base_model.layers[:-40]:
28     layer.trainable = False
29
30 modeloB1.compile(optimizer=Adam(learning_rate=1e-5),
31                  loss='binary_crossentropy',
32                  metrics=['recall'])
33
34 historyTransferLearning = modeloB1.fit(
35     X_train, Y_train,
36     epochs=15,
37     validation_data=(X_val, Y_val),
38     class_weight=dict_weights,
39     batch_size=32,
40     callbacks=[callback, reduce_lr]
41 )

```

5.1. Estrategias para combatir el Overfitting y el desbalance de clases

Para combatir el Overfitting utilizaremos la tecnica del Dropout que consiste en desactivar aleatoriamente el 40% de las neuronas de una capa durante el entrenamiento.

Además utilizaremos una función de pesos para combatir el desbalance de clases. Esta función se encarga de penalizar más a las clases con menos imágenes.

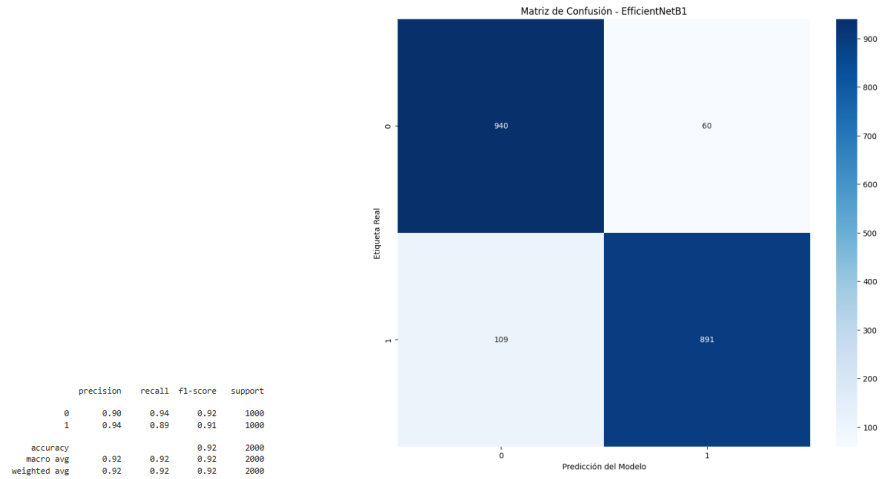
Y por último añadiremos un EarlyStopping y un callback para reducir el learning rate cuando el modelo deje de mejorar y para detener el entrenamiento si el modelo no mejora durante unas epocas determinadas..

5.2. Resultados

Tras entrenar el modelo utilizando las estrategias mencionadas anteriormente, obtuvimos los siguientes resultados utilizando los datos de prueba:

El modelo de Transfer Learning utilizando el EfficientNetB1 es el que mejores resultados da de los 3. Esto es debido a tres razones:

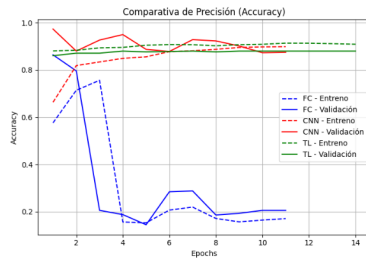
- Pre-conocimiento: Debido a que este modelo cuenta con conocimiento anterior basado en el dataset ImageNet, puede aprender características más complejas y generalizables.
- Entrenamiento propio: A pesar de tener un modelo funcional, hacemos cambios para que se adapte mejor a nuestras categorías.



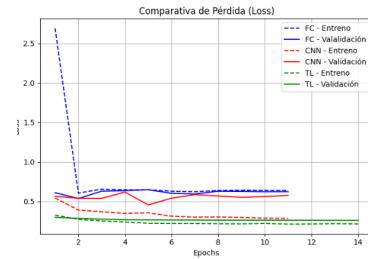
- Manejo de sobreajuste: Al tener las capas base congeladas, el modelo tiene muchísimos menos parámetros que aprender al principio. Esto evita que se sobreajuste o se aprenda de memoria el ruido de los datos.

6. Comparación entre los modelos

Aquí tenemos una comparación entre los tres modelos:



(a) Resultados Recall



(b) Resultados Perdida

6.1. Transfer Learning (TL)

El modelo de **Transfer Learning** demostró ser el más robusto y eficiente para la tarea de clasificación de imágenes de melanoma:

- **Desempeño:** Alcanzó el mayor nivel de precisión ($> 90\%$) y la pérdida más baja.
- **Generalización:** La convergencia entre las curvas de entrenamiento y validación indica una ausencia significativa de *overfitting*.

- **Estabilidad:** Presenta curvas suaves, lo que sugiere un aprendizaje sólido desde las primeras épocas.

6.2. Red Neuronal Convolutiva (CNN)

La arquitectura CNN nativa muestra un desempeño competitivo pero con limitaciones:

- **Inestabilidad:** Se observan fluctuaciones agresivas en la precisión de validación, lo que indica sensibilidad a los datos de prueba.
- **Precisión:** Aunque logra un buen aprendizaje (aprox. 88 %), no alcanza la eficiencia del modelo pre-entrenado.

6.3. Fully Connected (FC)

El modelo basado únicamente en capas densas resultó inadecuado para este dominio:

- **Colapso:** La precisión cae drásticamente después de la tercera época, estabilizándose cerca del 20 %. Esto se debe a que las capas densas ignoran la correlación espacial de los píxeles, esencial para identificar estructuras morfológicas en lesiones cutáneas.
- **Limitación Teórica:** La incapacidad de captar patrones espaciales en imágenes de alta complejidad provoca que el modelo converja en mínimos locales no óptimos.

7. Conclusiones

El mejor modelo es el Transfer Learning ya que es el que mejores resultados da y su curva de validación es la más plana, lo que aumenta la confianza en que funcionara correctamente incluso con datos nuevos que no haya visto antes.